

C Programming

3_functions_p1

Functions

Функция — это именованный блок кода для решения задачи, который можно повторно использовать

Функции позволяют:

- **Избегать повторения кода.** Написали блок кода один раз - используете его везде сколько угодно раз
- **Структурирование.** Программу можно разбить на логические части, увеличивает читаемость.
- **Проще искать и исправлять ошибки** в небольших блоках (функциях) чем бегать по всей программе

```
returnType functionName(arguments) {  
    // function body  
    return (value);  
}
```

```
типВозвращаемогоЗначения имяФункции(Аргументы)  
{  
    // Тело функции  
    return (Возвращаемое значение);  
}
```

- **Тип возвращаемого значения**
 - int - вернет целое число;
 - float/double - вернет вещественное число;
 - void - ничего не вернет;
- **Имя функции** - имя нашего блока кода;
- **Аргументы** в круглых скобках, это входные данные в наш блок, могут быть пустыми;
- **Тело функции** в фигурных скобках {}. Это и есть блок кода с инструкциями;
- **return** – вернуть. Значение, которое функция отдаст как результат

```
int sum(int a, int b)
{
    return a + b;
}
```

Имя функции – **sum**, возвращает тип данных **int**.

На вход принимает два **int** аргумента - **a** и **b**.

Тип данных прописываем для каждой переменной!

Function call

```
int result = sum(1, 2);
```

- Для вызова любой функции мы используем имя этой функции и передаем в круглых скобках аргументы
- Для получения возвращаемого результата в переменную пишем знак равно

```
int sum(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

...

```
printf("sum = %d\n", sum(1, 2));
```

...

return

```
int sum(int a, int b) {  
    return a + b; // Это выполним  
  
    int c = 5; // Сюда НИКОГДА не попадем  
    return c; // Сюда тоже  
    // Сюда тем более  
}
```

Самостоятельно: Написать простой калькулятор для двух чисел и знака операции, введенных с клавиатуры. Каждая операция `+`, `-`, `\*`, `/` и т.д. - отдельная функция. **Опционально:** использовать **switch-case** для выбора функции. Используйте подобный формат в `scanf()`: `"%d%c%d"`

Void

```
void meow(int n) {  
    for (int i = 0; i < n; i++) {  
        printf("meow\n");  
    }  
    return; // return здесь не обязателен  
}
```

```
...  
meow(5); // мяукает 5 раз  
...
```



- **Объявление (declaration)** функции - сообщаем компилятору кратко какое имя у функции и какие типы данных.

```
int sum(int a, int b); // заголовок функции (прототип)
```

- **Определение (definition)** функции - определяем что делает функция, т.е. все то, что содержит имя функции и тело функции - есть уже определение.

```
int sum(int a, int b) { // {} - есть тело ф-ии, значит это определение ф-ии  
    return a + b;  
}
```

Компилятор проверяет использование функций в исходном файле сверху вниз и если он найдет вызов функции раньше чем объявлена сама функция, то это может привести к неопределенному поведению программы. Компилятор в таких случаях делает **предупреждение - implicit declaration (неявное объявление)**

```
void woof(int n); // Объявление (definition)
```

```
int main() {  
    woof(5); // Вызывается до определения  
}
```

```
void woof(int n) { // Определение (declaration)
```

```
    for (int i = 0; i < n; i++) {  
        printf("meow\n");  
    }  
    return;
```

```
}
```

`max(a, b)`

*Используя тернарный оператор и `return` упростить тело функции `max` до 1 строки

Предикаты — это выражения или утверждения, которые могут быть либо истинными, либо ложными

```
int isEven(int a);
```

```
while (isBusy()) {  
    // ..  
}  
  
if (isValid()) {  
    // .. do smth  
}  
  
if (!isEmpty()) {  
    // ...  
}
```

Сеттеры/геттеры для установки/получений значений

```
setName();  
setGenre();
```

```
getMessage();  
getTimeOfDay();
```

Github

<https://github.com/kruffka/C-Programming>

